Note capabilities: repository fact-check

Overall assessment

The **main application under development**, rather than the separately documented older prototype, is available as source code in this repository. Its conversation can call real tools to read, search, create and revise workspace files. With configured model access, this supports turning supplied thoughts into notes, summarizing and comparing selected notes, drafting new text and updating saved content. It does not establish dependable discovery or complete analysis of a large collection, faithful speech recognition, or a working connection to original Apple Notes.

The reported experience of turning freely dictated thoughts into polished notes is consistent with the current technical path and with the historical prototype description. It was not reproduced here, and the report does not identify which variant or build was used. The prototype's current capabilities remain **UNCLEAR** because its source and running installation were not available in this checkout.

Scope and shared foundations

Product variants

- **Main application:** `client/app/`, its native host and `client/crates/`. The assessments below primarily concern macOS and accessible workspace text files. Shared code is not native iOS qualification.
- **Earlier prototype:** only the historical account in Prototype handover is available. Its cited Swift application and PiBridge sources are explicitly outside this repository. Its findings and old test results are secondary evidence, not checks performed for this report.
- **Design workshop:** `client/workshop/` contains separate interface scenarios with synthetic behavior. An Apple Notes example there does not establish an operational connection.

The root README's statement that chat and product actions are unavailable is inconsistent with the inspected implementation. The following assessments use the implemented conversation/tool path, not that outdated description. [E1, E8]

How content reaches the model and becomes a note

1. **Supplied content:** users can type or paste text, attach supported text files, or select saved workspace files as conversation references. Text/Markdown/JSON attachments and PNG/JPEG/GIF/WebP images are supported; images require a compatible model. PDF, Word and arbitrary rich-note formats are not native text attachments in this path.
2. **Saved workspace content:** the model is offered directory listing, filename search, literal text search, text/image reading, file history, and previous submitted conversations in the same workspace. It can call those tools during the conversation; it does not automatically receive every note. Meaning-based reasoning happens in the model after content is obtained.
3. **Output:** an answer can remain in the saved conversation. A separate note requires a successful file-write operation, not merely an answer saying it was saved. The write/edit path checks the file version, stores content and records history; it can report partial failures rather than falsely acknowledging full success.
4. **Formatting:** ordinary Markdown supports paragraphs, headings, lists and nested items, emphasis, quotations, links, tables and code. This is not preservation of another application's full rich-text layout. The

chat instruction excludes generated HTML, images, mathematical notation rendering and diagrams; new text and tables are the established output route. [E1–E6]

Sources must remain distinct

Source	What the inspected main application supports
Text supplied directly	Model processing without a note-app connection; saving a separate result requires workspace write access.
Workspace text files, including imported copies	Reading, model processing and version-checked editing. A writable workspace and working model connection are prerequisites.
Original Apple Notes	**UNCLEAR.** No bundled Notes-specific adapter was established in the inspected main application. A real generic macOS shell/process route can potentially run source-specific automation; its presence does not prove Notes permission handling, reliable reading, writing, title/folder changes or format preservation.
One-time imported copies	Native publication of prepared text is implemented. Obtaining and converting a particular external format still needs a working source-specific route or supplied export. Later edits to the original are **not automatically transferred by this import mechanism**, and copy edits are not written back.

Plugins are actually connected to ordinary conversations, but only available, enabled tools enter a model request. A private installation might contain additional plugins; none were inspected. The import instructions ask the agent to leave originals unchanged, but the general shell is not a technical read-only lock. The historical prototype's restrictions on shared, locked and richly formatted Notes must not be assumed for the main application's general-purpose scripting route. [E7, E8]

Collection size, freshness and evidence quality

The built-in filename and text searches match literal text, not meaning; text search is case-sensitive. The model can try related words, inspect folders and read candidate notes, so the absence of a dedicated semantic index does **not** make contextual retrieval impossible. Conversely, a search result is not proof that all relevant notes were considered.

Files and search results arrive in limited portions, so the assistant may need several calls to inspect a note or folder. Larger investigations are possible, but the model cannot keep an unlimited amount of source text in view; older results can be omitted or summarized. Some file types and technical entries are not included in ordinary text searches. A few selected notes are therefore a much stronger basis for an answer than an unverified claim to have examined the entire collection; exact technical limits are recorded in the evidence section. [E2]

A selected current-file reference resolves when submitted; a historical reference fixes an earlier revision. Ordinary searches and successive reads are not a frozen, collection-wide snapshot. Later source changes require another read; imported copies do not establish the current state of their originals. [E2, E5, E7]

Ratings describe technical feasibility, not measured answer quality. No live model was asked to demonstrate faithful rewriting, correct classification, contradiction detection or valid pattern inference. Successful storage tests do not establish those qualities. No personal notes were used or changed.

Overview of N01–N10

YES means technically possible through an inspected, connected path. **PARTLY** means the complete proposed experience exceeds the supported scope. **NO, CURRENTLY** identifies a concrete missing route. **UNCLEAR** identifies missing evidence rather than impossibility.

The main rating is for **internal text notes on macOS**. The external-source and prototype columns are separate assessments, not deductions from the main rating. The common reasons for their UNCLEAR ratings are explained above; no complete capability is rated NO merely because a named menu item or search result is absent.

ID and capability	Main application: internal notes	Main application: original Apple Notes	Earlier prototype, current verification	Main reason or prerequisite
N01 — Develop thoughts	**YES**	**UNCLEAR**	**UNCLEAR**	Model processing plus saved Markdown; integrated dictation needs an Apple Silicon Mac, speech resources and microphone permission.
N02 — Place content appropriately	**PARTLY**	**UNCLEAR**	**UNCLEAR**	Read and edit a suitable note; finding the right destination across an unfamiliar collection inherits the retrieval limits.
N03 — Retrieve content	**PARTLY**	**UNCLEAR**	**UNCLEAR**	Literal search plus model-led exploration; contextual discovery is possible but collection-wide coverage is not established.
N04 — Answer questions about notes	**YES**	**UNCLEAR**	**UNCLEAR**	Read one or several notes and answer with filenames, quotes and available line references.
N05 — Condense content	**YES**	**UNCLEAR**	**UNCLEAR**	Summarize supplied or retrieved text; save the summary if requested.
N06 — Compare and combine	**YES**	**UNCLEAR**	**UNCLEAR**	Read selected notes, compare their text and publish a combined result.
N07 — Identify patterns and connections	**PARTLY**	**UNCLEAR**	**UNCLEAR**	Evidence-linked interpretations of selected notes are possible; dependable collection-wide pattern discovery is not established.
N08 — Transform content	**YES**	**UNCLEAR**	**UNCLEAR**	Produce and save new text, outlines and Markdown tables; not arbitrary finished visual formats.
N09 — Continue and revise notes	**YES**	**UNCLEAR**	**UNCLEAR**	Version-checked replacement or exact edits, with history and conflict protections.
N10 — Organize the collection	**YES**	**UNCLEAR**	**UNCLEAR**	File/folder actions are connected to the interface; macOS chat can also use shell commands.

iOS differences: integrated dictation and the inspected shell/process route are **NO, CURRENTLY** on iOS because their capability gates explicitly disable them. Text-processing foundations are shared, but no iOS device journey was run. N10's folder operations remain available in the shared file interface when storage is writable; equivalent chat-driven folder restructuring cannot be inferred from the macOS shell path. These are platform-specific product gaps, not evidence that the entire note capability is impossible. [E4, E6, E7]

Explanation by capability

N01 — Develop thoughts: YES

Users can ask for supplied thoughts to become clearer sentences, paragraphs, headings and lists, then have the result saved as a Markdown note. On an Apple Silicon Mac, recording is locally transcribed, inserted into the editable chat draft and saved as a draft; the user sends it before the model processes it and writes a note. This chain is connected in code, but neither recognition accuracy nor faithful preservation of meaning was demonstrated with a real recording/model. Model access, speech resources and microphone permission are setup prerequisites; iOS integrated dictation would require product work, and saving into original Apple Notes still needs access verification. [E1, E3, E4, E6]

N02 — Place content appropriately: PARTLY

Once a destination note is identified, the model can read it, interpret a new thought and insert text into the relevant passage. No separate categorization feature is needed: the ordinary read/edit tools provide the necessary execution path. Finding the best destination from meaning alone across an unfamiliar collection is less established, because candidate discovery relies on literal searches and model-led reading rather than a demonstrated complete selection process. Naming a note or narrowing the relevant folder helps immediately; a representative placement-quality check should precede claims of reliable autonomous filing. [E1–E3]

N03 — Retrieve content: PARTLY

Users can describe what they remember without knowing a filename, and the model can search related words and inspect candidate files. It can also enumerate and read a small collection, interpreting meaning rather than merely repeating search matches. Literal, case-sensitive search can miss differently expressed ideas, and successive pages are neither a completeness guarantee nor a stable snapshot of changing files. The next need is a retrieval check with known relevant notes and paraphrased questions; a more systematic discovery mechanism is a possible product improvement, not a prerequisite imposed by this assessment. [E1, E2]

N04 — Answer questions about notes: YES

The model can read one or several supplied, selected or discovered notes and use them in an answer. Paths, text, revision information and search-result line numbers allow it to name and quote supporting passages; selected file references can also be reopened from the conversation. This does not provide automatic verification that every claim is correctly cited, nor does an input-file reference prove that it supports a particular sentence. Asking explicitly for filenames and quotes is possible now; source-to-claim accuracy and user-friendly citation navigation still need practical evaluation. [E1, E2, E5]

N05 — Condense content: YES

Users can request a summary, key points or a shorter version of supplied or saved notes. The model receives the text through the connected input/read path, and can return the summary in chat or write it as a separate note. Long notes must be read in full through successive pages, while a collection summary additionally depends on selecting the right notes. This needs configured access rather than a dedicated summarization feature; preservation of important statements remains a live-model test gap. [E1–E3]

N06 — Compare and combine: YES

Users can choose several notes and ask for similarities, differences, apparent contradictions and a combined draft. Reading the sources and writing the combined text are supported, so no separate comparison module is necessary. Whether a difference is a genuine contradiction, a later revision or a different context must be

interpreted from the available evidence; source dates or history can help when present. Selected-text processing needs no new product feature, but comparison quality and coverage of an unspecified entire collection have not been demonstrated. [E1–E3, E5]

N07 — Identify patterns and connections: PARTLY

The model can examine several available notes, propose recurring themes or developments and quote passages supporting each interpretation. That selected-note workflow is technically possible with existing tools, provided observations and interpretations are explicitly separated in the request and result. Complete collection-wide discovery is a stronger claim: missing candidates, omitted context and missing dates can hide counterexamples or create misleading trends. A representative evidence-and-counterexample check is the priority; dependable broad analysis may also need an explicit coverage-tracking workflow. [E1, E2, E5]

N08 — Transform content: YES

Users can turn supplied or retrieved knowledge into a new draft, outline, checklist or Markdown table and save it. This uses ordinary model output and file creation rather than a separately named transformation feature. Finished slide decks, image-based diagrams and arbitrary rich-format exports are not established by this text path, and existing source attachments are not automatically understood merely because their links appear in a note. Text transformations need model access; additional output formats need a suitable converter or extension and its own verification. [E1–E3, E6]

N09 — Continue and revise notes: YES

The assistant can read a saved note, add information, revise selected passages or replace its text. Exact edits must match uniquely, and a stale file version or active editor draft can block overwriting; successful changes retain file history. These checks protect storage consistency, not whether the new wording is factually correct, and very large notes exceed the agent-write limit. Imported copies can be changed this way, but changing an Apple Notes original or reconciling later original edits requires a separately verified source workflow. [E3, E7]

N10 — Organize the collection: YES

Users can name files, create folders, move or rename notes and develop a folder-based structure through the connected file interface. On macOS, the assistant can also perform such organization through its actual shell tool; editing headings inside notes uses ordinary text edits. Shell actions have broader access and different safeguards than exact edits, and there are no dedicated built-in chat tools for rename/move/mkdir, so the macOS route must not be promised on iOS. Internal titles and paths are supported, but Apple Notes titles, accounts and folders need separate implementation/access verification; dependable automatic organizational choices also remain unmeasured. [E2, E3, E6, E7]

Evidence and verification

Inspected version

- Branch: `use-case-validation`.
- Commit: `27b6831c3e55b2de4a59d6cd32cf1935d373338d`.
- Initial working tree: clean. No local product-code differences were used. The only intended tracked-content change is this report; installed dependencies and test outputs are ignored development artifacts.
- Prototype handover provenance, **not a freshly inspected prototype revision**: `main`, `53863aa6be03a1033d8da65801f84399799a5a0c`, version 0.5.0/build 22. The handover discloses a modified

embedded bridge and a dependency-version mismatch, further limiting reproduction from that document alone.

Shared code evidence

Paths and line ranges refer to the inspected commit. E1–E7 are source inspection unless a test execution is explicitly listed below.

Evidence	Inspected files and relevant lines	Supports
E1 — Reachable conversation/model/tool chain	`client/app/src/ai/chat.ts:874–940`; `client/app/src/ai/native-conversation.ts:284–291`; `client/app/src-tauri/src/ai_conversations.rs:269–299,630–672,866–894,1785–1837`; `client/crates/situate-agent/src/execution.rs:406–455,478–582`	Submission reaches the native owner, offered tools reach the model, calls execute and results are retained. All N01–N10 rely on this chain.
E2 — Discovery and size/coverage limits	`client/app/src-tauri/src/ai_builtin_tools.rs:9–79`; `client/app/src-tauri/src/file_tool.rs:215–307`; `client/app/src-tauri/src/files.rs:1501–1563`; `client/crates/situate-core/src/content/mod.rs:1026–1058`; `client/crates/situate-agent/src/compaction.rs:108–150`; `client/crates/situate-agent/src/execution.rs:455–459`	Literal matching, pages/excerpts, format limits, exclusions, context reduction and model-call guard. Especially N02–N07.
E3 — Durable content changes	`client/crates/situate-core/src/content/agent_tools.rs:10–77`; `client/app/src-tauri/src/files.rs:1430–1500`; `client/app/src-tauri/src/ai_conversations.rs:345–413`	Exact edits, version checks, draft protection, file/history acknowledgments and separate conversation persistence. N01, N02, N05, N06, N08–N10.
E4 — Speech to editable input	`client/app/src-tauri/dictation/Sources/SituateDictation/Dictation.swift:174–314`; `client/app/src-tauri/src/dictation.rs:198–224`; `client/app/src/dictation/draft.ts:14–49`; `client/app/src/components/ChatPane.vue:858–873`	Microphone/recognition path, platform restriction and draft insertion rather than automatic submission. N01.
E5 — References and inspectable sources	`client/app/src-tauri/src/ai_conversations.rs:630–652`; `client/crates/situate-core/src/content/live_references.rs:38–144`; `client/app/src/components/ChatPane.vue:455–495`; `client/app/src/components/chat/ChatToolCard.vue:145–168`; `client/app/src/components/chat/ChatMarkdown.vue:13–20`	Input reference identity, current/historical resolution, visible tool results and ordinary answer rendering—not automatic claim-level citation validation. N03, N04, N06, N07.
E6 — Formatting and organization	`client/crates/situate-core/src/rendering.rs:6–88`; `client/app/src/ai/attachment-runtime.ts:9–38`; `client/app/src-tauri/src/ai_conversations.rs:1146`; `client/app/src/files/FileEditor.vue:918–924,953–1024`; `client/app/src-tauri/src/files.rs:1030–1053`; `client/app/src-tauri/src/shell_host.rs:32–57`; `client/app/src-tauri/src/ai_builtin_tools.rs:107–119`	Markdown rendering, supported inputs, output instructions, UI folder operations and capability-gated chat shell. N01, N08, N10.
E7 — External access versus copy publication	`client/app/src-tauri/src/ai_conversations.rs:585–605`; `client/app/src-tauri/src/ai_executor.rs:1157–1207`; `client/crates/situate-core/src/plugins/runtime.rs:341–376`; `client/app/src-tauri/src/plugin_host.rs:800–845,878–893`; `client/app/src-tauri/src/ai_onboarding.rs:412–416`; `client/crates/situate-core/src/content/import_batch.rs:8–18,106–115`; `client/app/src-tauri/Info.macos.plist:3–4`; `client/app/src-tauri/Entitlements.macos.plist:3–5`	Available workspace plugin tools, generic process/HTTP execution, source-preservation instructions, prepared-copy publication and iOS process limits. macOS configuration supplies microphone declarations, not a qualified Notes permission flow. Applies to external-source qualifications for every capability.

Technical bounds from E2, E3 and E5: text pages are at most 32 KiB, search/list pages 200 entries and matching-line excerpts 1 KiB. The normal read tool rejects files above 16 MiB; agent text writes are limited to 1 MiB per file; explicitly attached workspace references have a combined 128 KiB input limit. The execution loop has a 128-model-call guard. Non-UTF-8 files are not searched as text; ordinary listings exclude technical/ignored entries and symbolic links. These limits do not measure successful collection size or semantic completeness.

Documentation and historical evidence — not current execution

E8: `docs/prototype-handover.md:3–19,39–58` explicitly separates the prototype and reports earlier test limits; `:225–243` describes dictation, and `:334–354` describes live Apple integrations. That account supports the plausibility of the reported usage and describes simple-text Notes writes, shared-note read-only access, locked-note exclusion and restrictions on rich content. It does not establish those behaviors in the main app or in the user's current prototype build. `docs/development/workspace-development-plan.md:280–286` explicitly leaves Notes access, permissions and distribution qualification open. `client/workshop/src/onboarding/source-conversation.test.ts:30–42` checks scripted interface copy, not real Notes access. `README.md:5` supplies the outdated status statement discussed above.

Checks actually executed

All commands used the repository's Volta-pinned Node/npm and coordinated command queue. Rust used the pinned 1.98.1 toolchain. Tests used synthetic content or owned temporary files, not personal notes.

Check	Result	What the result does and does not show
Product Vitest: `src/dictation/runtime.test.ts`, `src/ai/chat.test.ts`, `src/ai/native-conversation.test.ts`, `src/ai/content-references.test.ts`, `src/files/FileEditor.test.ts`	**144 passed, 5 files**	Draft insertion, conversation actions, references and file UI behavior with simulated native/file responses. Not real transcription, native storage integration or live model quality.
Rust core: `content::agent_tools::tests`	**4 passed**	Real temporary-file writes/history, exact-edit matching, stale-version/draft protections and failure recovery.
Rust core: `content::import_batch::tests`	**16 passed; 1 benchmark ignored**	Prepared-copy publication, retained text/metadata/link behavior, limits and partial failures. Not an Apple Notes extraction/conversion test.
Rust core: `content::live_references::tests`	**8 passed**	Current/historical references, moves, deleted/reused paths and size limits using real temporary storage.

Reproduction commands, from the repository root with Volta shims first on `PATH`:

```
node tools/run-coordinated.mjs -- npm --prefix client run test \
  --workspace @situate/client -- src/dictation/runtime.test.ts \
  src/ai/chat.test.ts src/ai/native-conversation.test.ts \
  src/ai/content-references.test.ts src/files/FileEditor.test.ts --allowOnly=false

# Run separately for each FILTER listed in the Rust rows above.
node tools/run-coordinated.mjs -- node tools/run-coordinated.mjs --resource build -- \
  rustup run 1.98.1 cargo test --manifest-path client/Cargo.toml \
  -p situate-core --locked FILTER -- --test-threads=4
```

An initial Rust filter named `content::live_reference_tests` selected **zero tests** and supplies no evidence; it was corrected to the eight-test filter above. The literal-search test at `client/app/src-tauri/src/file_tool.rs:518–565` and the native ordinary-chat/import journeys were inspected, not executed.

Not performed: live model requests, microphone recognition, a native application journey/build, macOS Notes permission/access tests, rich-note round trips, iOS device tests, or a collection-wide semantic-quality evaluation. An isolated real-model/microphone/Notes test environment was not established for this review; no private model credentials or note stores were inspected to obtain one. Historical prototype tests were not rerun. These gaps leave original-source access and current prototype ratings UNCLEAR; they do not negate the connected internal-text workflows.

Three priority next steps

1. **Identify and reproduce the reported workflow.** Record the actual variant/build and note destination, then use synthetic dictated thoughts to verify transcript, model rewrite and persisted note separately. Review content omissions/additions, not only whether saving succeeds.
2. **Qualify Apple Notes separately from imports.** On the intended macOS distribution, test permission grant/denial, selected accounts/folders, simple and rich notes, shared/locked notes, titles, attachments and allowed writes. Verify that a later original edit leaves a copy unchanged unless an explicit reconciliation route exists; assess iOS independently.
3. **Evaluate retrieval and reasoning on a known synthetic collection.** Include paraphrases, similarly named notes, conflicting statements, dated changes and counterexamples. Check destination choice, missed notes, citations and distinction between observation and inference; use failures to decide whether clearer scope, better discovery or coverage tracking is needed.